external-watchdog-for-forced-logout-architectural-gap

External Watchdog for the Forced-Logout Architectural Gap — Proposal

Revision: 1 **Last modified:** 2026-08-19T00:00:00Z **Status:** proposal (Phase 1 design only — NOT implemented, per task scope) **Authority:** SDD task #85, filed against the gap exposed by incident #3 (2026-08-18 23:45:49 forced-logout, `docs/qa/BOB-120/`) in the preventive timer landed by BOB-116 / task-77.

1. Problem statement

BOB-116 (the 2nd forced-logout incident, 2026-08-18 20:50:59) produced a proactive 5-signature detector — `challenges/scripts/resource_pressure_signature_challenge.sh` — and task-77 wired it into two standing enforcement points: `scripts/pre_build_verification.sh` invariant 25 (build-time only), and an hourly `systemd --user` timer, `boba-resource-pressure-check.timer` (`scripts/systemd/user/boba-resource- pressure-check.{service,timer}`, installed via `scripts/install-resource-pressure- timer.sh`).

That timer has a structural blind spot: **it runs inside `user@1000.service`, the exact resource pool it exists to monitor.** When `user@1000.service` is SIGKILLed, systemd tears down its entire cgroup subtree — including the timer, its service unit, and every process the timer would have used to detect the precursor condition. The detector cannot fire *during* the failure it is meant to catch, because it dies with everything else.

Incident #3 forensics (2026-08-18 23:45:49, `docs/qa/BOB-120/journalctl_23-42_ to_23-46.log`) prove this is not theoretical. Verified directly from that capture:

```
23:45:49  systemd[1]: user@1000.service: Main process exited, code=killed, status=9/KILL
```

```
23:45:49  systemd[1]: user@1000.service: Killing process <PID>
(jackett) with signal SIGKILL.
23:45:49  systemd[1]: user@1000.service: Killing process <PID>
(claude) with signal SIGKILL.
          [... 70+ more processes, every UID-1000 process on the
host, without
          exception ...]
23:45:50  systemd[1]: user@1000.service: Consumed 3h 3min 5.492s
CPU time,
          27.5G memory peak, 216.2M memory swap peak.
23:49:00  gdm-password: Session opened for milosvasic  ← operator
re-login,
          3m11s after the kill, the earliest point ANY UID-1000
process
          (including the timer) could run again
```

Per the task brief's own timeline, the timer had fired at 22:42 and 22:57 and was next due around 23:42:38 — inside the dead window. It never fired. The gap the brief describes ("fires, then blocked until 23:49 restart") is exactly what this capture shows: nothing owned by UID 1000, including the detector, exists between 23:45:49 and 23:49:00.

This is now a **2-for-2 pattern**: the 2026-07-07 incident produced §12.12 (thread-limit awareness); BOB-116 produced the resource-pressure detector + timer; and that timer's very first live incident window (23:45:49) demonstrates it cannot observe the failure mode it targets, because the failure mode is defined as *the death of the pool the timer lives in*. Task #85 is filed to design — Phase 1 only, no implementation — a watchdog mechanism that is structurally outside that pool.

2. Phase 1: root-cause the architectural constraint

2.1 Where the timer actually lives

```
$ systemctl --user show -p ControlGroup
ControlGroup=/user.slice/user-1000.slice/user@1000.service
```

Every unit `systemctl --user` manages — the resource-pressure timer included — is a child of `user@1000.service`. That unit is itself a **system-level** systemd unit (`user@1000.service`, started by PID 1 as UID 1000's "systemd –user" manager), and per the incident log, when systemd decided to stop that unit, it issued `SIGKILL` to every process in its cgroup subtree — the timer's own service, the challenge script it would run, and the shell that would have read the result — with no exception carved out for a monitoring process.

2.2 Rootless-mandate scope check (§11.4.161)

> "every project MUST use Podman in rootless mode … for **ALL containerized workloads**. Docker in rootful mode, sudo, or any escalation to root is FORBIDDEN unless the target platform has no rootless option…"

§11.4.161 is explicitly scoped to **containerized workloads**. A plain systemd unit or a cron entry is not a container, so a non-container watchdog does not trip the letter of this anchor. It *does* still cross this project's broader, otherwise-universal habit of never touching root on this host — a habit this document treats as a real cost to weigh, not a rule this design can silently route around (see §5).

2.3 Hard Stop #3 scope check

> "Container orchestration is owned exclusively by the project's own binary/orchestrator, `start.sh` … Direct `docker`/`podman start|stop|rm` and `docker-compose up|down` are prohibited as workflows."

This binds anything that starts/stops **boba's own containers**. It has no bearing on a systemd unit or cron entry that does not touch podman/docker at all. It becomes directly relevant only for the container-shaped variant of Option C/D below, where it would require the watchdog's own container lifecycle to be wired through `start.sh`, not raw `podman` calls.

2.4 The actual cgroup boundary that matters

The literal question Phase 1 has to answer is not "root vs. rootless" in the abstract — it is: **which systemd delegation boundaries on this host are structurally independent of `user@1000.service`'s cgroup subtree, so that a process living there is NOT torn down when `user@1000.service` is?**

Verified directly on this host (`nezha`, ALT Linux, single operator):

```
$ cat /proc/self/cgroup                    # this shell's own scope
0::/user.slice/user-1000.slice/user@1000.service/app.slice/tmx-boba-0404.scope

$ systemctl show crond -p Slice
Slice=system.slice

$ cat /proc/1530/cgroup                    # crond, the system cron daemon
0::/system.slice/crond.service
```

```
$ systemctl cat crond.service
[Unit]
Description=Vixie Cron Daemon
After=syslog.target systemd-user-sessions.service
[Service]
ExecStart=/usr/sbin/crond -n
[Install]
WantedBy=multi-user.target
```

`crond.service` has been running continuously in `system.slice` since boot (11:08:27), unaffected by both the 20:50:59 and the 23:45:49 kill — it is a **pre-existing, already-running, root-owned system service**, structurally sibling to `user.slice`, not a descendant of it.

A rootless-podman correction the brief's Option C description gets wrong. The brief frames Option C as "a podman container … is its own scope, survives user@1000.service SIGKILL." The incident log directly refutes this on this host's actual configuration:

```
23:45:49  systemd[1]: user@1000.service: Killing process <PID>
(jackett) with signal SIGKILL.
23:45:49  systemd[1]: user@1000.service: Killing process <PID>
(qbittorrent-nox) with signal SIGKILL.
23:45:49  systemd[1]: user@1000.service: Killing process <PID>
(s6-supervise) with signal SIGKILL.
23:45:49  systemd[1]: user@1000.service: Killing process <PID>
(conmon) with signal SIGKILL.
```

`jackett` and `qbittorrent-nox` are boba's own rootless-podman-managed containers. They were killed in the same sweep as everything else, because rootless podman for UID 1000 fundamentally runs *through* that UID's cgroup delegation — there is no rootless configuration on this host where a container owned by UID 1000 lives outside `user.slice/user-1000.slice`. A container-shaped watchdog only escapes this pool if it runs under a **different UID** with its own persistent `user@<uid>.service` — which is not a third independent design, it is Option D wearing a container. §2.5 below folds the two together for that reason.

2.5 Conclusion of Phase 1

There are exactly **three** structurally independent escape routes from `user@1000.service`'s cgroup subtree on a standard single-operator Linux host:

1. **`system.slice`, root-owned** — a *new* systemd unit installed as root (Option A), or **pre-existing** root-owned infrastructure this host already runs continuously and never needs re-installing (`crond.service` — Option B).

2. **A second UID's own `user@<uid2>.service`**, kept alive via `loginctl enable-linger` — independent of UID 1000's pool entirely (Option D; the corrected form of Option C).
3. **Off-host** — a genuinely separate machine (Option F).

Kernel-level BPF (Option E) is a fourth theoretical route (kernel space is outside every UID's slice by definition) but is explicitly out of this project's toolchain scope (§3).

3. Design options (A–G), enumerated with real evidence

Option A — New system-level systemd timer (root)

Mirror the existing `.service`/`.timer` pair, install under `/etc/systemd/system/` with `User=milosvasic` in the `[Service]` block (the unit *lives* in `system.slice`; the script it runs still executes unprivileged as UID 1000 — the escalation is confined to the one-time unit installation, not to the check itself).

- **Pros:** smallest design delta from what already exists (task-77's unit files are a near-direct template); systemd-native scheduling/logging (`journalctl -u`, `systemctl status`); structurally correct — genuinely outside `user@1000.service`.
- **Cons:** requires a **new, standing root-owned unit** the operator must install with `sudo systemctl daemon-reload` + `sudo systemctl enable --now`. Does not violate the *letter* of §11.4.161 (scoped to containerized workloads — §2.2), but it is a real, novel privilege-escalation surface on a host with two prior forced-logout incidents and CONST-033's standing host-safety history. That history is exactly why this class of change needs explicit sign-off (§5), not an assumption that "not a container" is the end of the analysis.

Option B — User crontab entry, reusing the existing `crond.service`

Add one line to `crontab -e` (as user `milosvasic`, no `sudo`) that invokes the **same, already-landed** `challenges/scripts/resource_pressure_signature_challenge.sh` on a fixed interval.

- **Pros:** `crond.service` is pre-existing, already running, already in `system.slice`, verified alive and unaffected through both incidents (§2.4) — **zero new standing root-owned service is created.** Installing the crontab entry itself needs no `sudo` (confirmed: no `/etc/cron.allow` restriction, `crontab -l` for `milosvasic` works unprivileged). Reuses the exact detector script byte-for-byte — no new detection logic. Classic, well-understood cron behaviour: a job's process tree stays under

`crond`'s own process group unless something explicitly re-parents it into a login session (traditional vixie-cron, as installed here — `vixie-cron-4.1. 20060426-alt10.3` — does not register a systemd-logind session for a plain cron job the way `sshd`/`gdm` do), which is the standard reason cron jobs are famously invisible to per-user systemd/cgroup accounting.
- **Cons:** cron's execution environment is minimal (no `PATH`, no `HOME` guarantees) — the detector script must be invoked with explicit absolute paths and an explicit environment, a well-known but mechanical adaptation. Less rich native logging than a systemd timer (no `journalctl -u` equivalent; needs its own log redirection, which the existing challenge script's evidence-file convention already provides). **Whether this specific ALT-Linux vixie-cron build ever invokes `pam_systemd` for a job run — which would matter if it turned out to move the job into a `user-1000.slice/session-N.scope` sibling of, rather than a descendant of, `user@1000.service` — is asserted here from cron's well-documented general behaviour, not measured live on this host. This is the single fact Phase 1.5 (a live spike, before any implementation) must confirm before Option B is implemented: schedule one cron job, capture its live `/proc/<pid>/cgroup`, confirm it is NOT a descendant of `user@1000.service`.**

Option C — Rootless-podman container watchdog (as originally framed)

Not independently viable on this host as described. §2.4 shows directly, from the incident's own journal, that boba's existing rootless-podman containers (`jackett`, `qbittorrent-nox`) are descendants of `user@1000.service` and were killed in the same sweep as every other UID-1000 process. A container run the normal way by UID 1000 gains no survivability over the existing systemd-user timer it would replace. The only way to make a podman-based watchdog survive is to run it under a different UID (folds into Option D) or run it rootful under a system-level unit (folds into Option A, minus the container's own added complexity for no additional benefit). Kept as its own row only to record this correction explicitly, not because it is a live candidate.

Option D — Separate user account watchdog (rootless, own `user@<uid2>.service`)

Create a dedicated low-privilege account (e.g. `boba-watchdog`), enable `loginctl enable-linger boba-watchdog` so its own `user@<uid2>.service` persists independent of any interactive login,

and run the detector (as a rootless systemd –user timer, or a rootless podman container per Hard Stop #3 wired through its own equivalent of `start.sh`) under that account.

- **Pros:** genuinely rootless day-to-day (the check itself never runs as root); structurally independent of `user@1000.service` by construction (different UID ⇒ different `user-<uid>.slice` subtree entirely); if the underlying incident mechanism ever turns out to target more than just UID 1000's slice, a second independent UID is the cleanest way to prove that boundary.
- **Cons:** heaviest option here. Needs a one-time `useradd` + `loginctl enable-linger` (root, one-time — same escalation *category* as Option A, slightly larger blast radius since it is a standing second account, not just a standing unit). Needs its own home directory, its own copy/clone of whatever the check depends on, and — the part this brief's framing leaves open — a cross-account notification path back to the operator's session (a shared log file both accounts can read is the simplest; anything richer needs its own design pass). On a single-operator laptop this is real ongoing account-management overhead for a survivability guarantee Option B already delivers using infrastructure that already exists and is already running.

Option E — Kernel-level BPF observer

Requires `CAP_BPF`/`CAP_SYS_ADMIN`-class kernel access for the tracepoint/ kprobe classes needed to observe an in-flight kill decision or memory pressure at the kernel layer; unprivileged eBPF on this host would still need `kernel.unprivileged_bpf_disabled` policy review and does not, in its unprivileged form, expose the process/cgroup-teardown hooks this problem needs. This is a materially different engineering discipline (kernel/BPF tooling this project does not currently maintain) and sits outside this project's userspace-orchestration model. **Not viable within this project's current scope and toolchain** — recorded per the brief's explicit instruction to acknowledge the kernel-scope boundary rather than hand-wave past it.

Option F — External monitoring host (Prometheus/ equivalent)

An exporter on a *separate* machine, polling this host, would be genuinely outside `user@1000.service` by construction — but it does not avoid the underlying problem, it relocates it: whatever runs the exporter/scrape-target **on this host** still has to live somewhere, and if it runs the normal way under UID 1000 it inherits the exact same failure mode this whole task exists to fix (§2.4). It would still need Option A/B/D underneath it just to

survive long enough to be scraped. It also requires infrastructure (a second always-on host, network exposure, credentials) this single-operator laptop environment does not currently have. **Disproportionate to the problem given B already closes it at near-zero cost** — not recommended, not ruled structurally impossible.

Option G — Accept the gap, document it

Honest fallback if none of A/B/D were justified relative to their cost. Given that Option B closes the structural gap using infrastructure that is already installed, already running, and requires **no new standing root service**, accepting the gap is **not** the right default here — it would be choosing to leave a real, now twice-demonstrated blind spot open when a near-free fix exists. G is recorded because the brief asked for an honest answer if it were the right call, not because it is.

4. Recommended path

Primary recommendation: Option B (user crontab, reusing the existing detector script and `crond.service`'s pre-existing `system.slice` presence), **kept alongside** — not replacing — the existing `boba-resource-pressure- check.timer` from task-77.

Rationale:

- It is the only option that closes the structural gap **without creating any new standing root-owned service or account** — it reuses infrastructure (`crond.service`) that is already installed, already enabled at boot (`WantedBy=multi-user.target`), and empirically proven to have survived both forced-logout incidents on this exact host.
- The day-to-day install action (`crontab -e`) needs no `sudo` from the operator or this project's automation.
- It reuses the exact, already-reviewed detector script from BOB-116/task-77 verbatim — no new detection logic, no new failure surface in the thing that decides "is the host under pressure."
- The systemd –user timer stays installed. When the host is healthy it still gives the richer, native `journalctl -u/systemctl status` visibility task-77 built; the cron entry is specifically the layer that survives the one scenario the timer cannot.

Escalation path, if Phase 1.5's live cron/cgroup check comes back adverse (i.e., if this specific vixie-cron build turns out to nest job processes under `user@1000.service` after all): fall back to **Option A**, since it is the next-cheapest structurally-correct option and reuses task-77's unit files almost unchanged. **Option D** stays on the table only if a future incident shows the kill mechanism

reaches beyond UID 1000's slice (which would also invalidate Option A/B) — there is no present evidence for that, so it is not part of this recommendation.

5. Operator decision required (§11.4.66)

This document is Phase 1 design only. Before any implementation proceeds, the operator needs to make an explicit choice, because every viable option here either (a) reuses a pre-existing root-owned daemon without installing anything new as root (Option B) or (b) installs new root-owned standing infrastructure (Options A/D) — and this host's incident history (2 forced logouts, CONST-033 host-safety criticality) means that distinction is not something this task should decide unilaterally.

Question for the operator:

1. Approve **Option B** (cron, reusing `crond.service`, no new root installation) as the implementation to proceed with, subject to Phase 1.5's live verification (§3, Option B "Cons") confirming the cron job's process tree is genuinely outside `user@1000.service`'s cgroup on this host?
2. If Phase 1.5 comes back adverse, or if the operator prefers not to depend on cron's env-handling quirks at all, pre-approve falling back to **Option A** (new root-owned systemd unit, `sudo` required once at install time) instead?
3. Any preference on keeping the existing `boba-resource-pressure-check.timer` running in parallel with whichever escape-route option is chosen (this document's recommendation is yes — belt-and-suspenders, richer logging while the host is healthy)?

No implementation proceeds on any option until this is answered.

6. Estimated implementation cost (ESTIMATES — need live verification)

These are effort estimates only. None of these options were implemented or timed in this task; every figure below is an estimate pending a real implementation pass, marked explicitly per the anti-bluff mandate.

Option	One-time privileged action	Estimated engineering effort	Notes
B (cron)	None (no `sudo` to install the crontab entry)	~30–60 min (install/uninstall helper + env-hardening	Phase 1.5 live cgroup check (~15 min) is a hard

Option	One-time privileged action	Estimated engineering effort	Notes
		wrapper around the existing script, mirroring task-77's mechanical-verification pattern)	prerequisite before this is trusted
A (system timer)	`sudo systemctl daemon-reload && sudo systemctl enable --now` (one-time)	~1–2 hours (adapt task-77's existing unit files + installer to `/etc/systemd/system/`, `User=milosvasic`)	Escalation path only, per §4
C (rootless container, as-framed)	N/A — not independently viable (§3)	N/A	Superseded by this correction; do not implement as originally scoped
D (second account)	`sudo useradd` + `sudo loginctl enable-linger` (one-time)	~1 day+ (account provisioning, script deployment under the new account, cross-account notification design)	Only revisit if a future incident shows the kill reaching beyond UID 1000's slice
E (BPF)	Kernel capability grant	Not estimated — outside current toolchain/ expertise	Explicitly out of scope
F (external host)	New host provisioning + network exposure	Not estimated — disproportionate for this environment	Not recommended
G (accept gap)	None	0	Not recommended given B's near-zero cost

7. Explicit non-implementation notice

Nothing in this document has been implemented. No crontab entry, no new systemd unit, no new user account, and no code change of any kind was made as part of this task. This is a Phase 1 design artifact only, per the task's explicit "DO NOT IMPLEMENT" constraint and §11.4.66 (operator decision required before any code lands).

8. Open items (tracked, not silently dropped — §11.4.197)

- **Phase 1.5 live verification (blocking, before Option B implementation):** schedule a single cron job on this host, capture its live `/proc/<pid>/cgroup`, and confirm it is NOT a descendant of `user@1000.service`. This document's Option B recommendation is conditional on that result.
- **Cross-account notification design** (only relevant if Option D is ever revisited): how a watchdog running under a second UID surfaces a finding back to the operator's session — not designed here, out of scope for Phase 1.
- **Remediation policy** (out of scope for this task): today's detector is purely detective/logging (per task-77). Whether an external watchdog should ever take a safe remedial action (e.g. reaping a known-pathological process pattern, as was done manually in BOB-116) is a separate design question this document deliberately does not answer.
- **Attribute-the-SIGKILL** long-run investigation (referenced from BOB-116's own followups) remains open and unconfirmed; this design does not depend on that root cause being resolved — it treats "the pool can die without warning" as the given constraint regardless of the exact trigger.

9. Cross-references

- §12.12 (RLIMIT_NPROC / process-thread-limit awareness) — the 2026-07-07 incident's anchor; this is the 3rd incident of a related resource-pressure class on this project.
- BOB-116 — 2nd forced-logout incident (2026-08-18 20:50:59), root cause §11.4.6 `UNCONFIRMED` for the exact SIGKILL trigger, contributing factors CONFIRMED; produced the detector this proposal reuses.
- `docs/qa/BOB-120/journalctl_23-42_to_23-46.log` — incident #3 raw evidence (2026-08-18 23:45:49), the direct forensic basis for §1–§2 of this document.
- task-77 report (`.superpowers/sdd/task-77-report.md`) — landed the detector + the `boba-resource-pressure-check.{service,timer}` pair this proposal builds on and recommends keeping.

- Task #85 — this design task, SDD-internal numbering (see `.superpowers/sdd/task-85-design-report.md`).
- §11.4.161 (rootless container runtime mandate) — scope-checked in §2.2.
- Hard Stop #3 (container orchestration owned by `start.sh`) — scope-checked in §2.3.
- §11.4.66 (interactive-clarification mandate) — the operator-decision gate in §5.
- §11.4.6 (no-guessing mandate) — governs the honest-boundary framing throughout, especially §3 Option B's cron/cgroup caveat and §6's cost estimates.